Amazon Audio Products: Data Analysis, Visualization, and Machine Learning Models

Name: Athul santo

Course: Data Science

Introduction

Project Overview

This project focuses on the **end-to-end analysis of Amazon audio products** by leveraging web scraping, data analytics, and machine learning techniques. The pipeline begins with extracting product data from Amazon, including details such as price, ratings, reviews, brand, and device type.

The collected data is then cleaned and transformed to perform **exploratory data analysis (EDA)** and build **interactive dashboards** that provide insights into market trends, customer preferences, and product distribution.

Further, the project implements multiple machine learning models, including:

- A **recommendation system** to suggest similar audio products to users
- A **device type prediction model** to classify products based on their features
- A **clustering model** to group products into meaningful segments

The final outcome of the project is a **data-driven system** that helps understand the Amazon audio product landscape and supports better decision-making for both customers and businesses.

Problem Statement

With the rapid growth of e-commerce platforms like Amazon, users are presented with a vast number of audio products, making it difficult to:

- Identify the best products based on their preferences
- Understand differences between product categories
- Make informed purchasing decisions

Ultimately, the goal is to create a **comprehensive analytical and predictive system** that simplifies product discovery and enhances decision-making in the audio product domain.

Project Objectives

The objective of this project is to analyze Amazon audio product data by collecting and preprocessing it for meaningful analysis. It also aims to identify trends using exploratory data analysis and present insights through interactive dashboards built with Power BI.

Additionally, the project focuses on developing machine learning models, including a recommendation system, device type prediction, and clustering, to generate useful insights and support better decision-making.

Data Collection (Scraping)

Scraping

This project uses **Python** as the core programming language due to its simplicity and strong support for automation and data processing.
Selenium WebDriver is used to automate browser actions and extract dynamic content that loads using JavaScript, while
ChromeDriver acts as a bridge to control the Chrome browser.
Pandas is utilized to organize, clean, and store the scraped data in a structured CSV format. Additionally, built-in libraries such as
os help manage files and resume the scraping process, **time** is used to introduce delays for proper page loading, and
random adds variable waiting times to mimic human behavior and reduce the chances of detection.
Overall, the scraping process works by automating a browser to search for audio products on Amazon using multiple keywords, navigating through multiple pages, extracting relevant product details, removing duplicates using ASIN, and storing the final dataset in a CSV file for further analysis.

Overall WorkFlow

```
Start
 ↓
Load existing data
 ↓
For each search term:
   For each page:
      Open Amazon page
      Handle CAPTCHA
      Extract products
      Remove duplicates
      Save periodically
 ↓
Stop when 1500 products collected
 ↓
Save final CSV
```

Data collected

ASIN
Unique product ID used to identify each item and remove duplicates
Product Title
Name and description of the product
Brand
Manufacturer name (usually taken from the title)
Price
Cost of the product in INR
Discount Percentage
Shows the offer or discount on the product
Rating
Customer rating out of 5
Review Count
Number of customer reviews
Category
Product category (fixed as “Audio”)
Product Link
Direct URL to the product page
Search Term
Keyword used to find the product

Data Preprocessing

Data cleaning:

Handling duplicate values
Data type conversion
Vectorization
Dropping
Handling null values

Feature engineering:

Has_mic- Extracted form title
Is_wired- Extracted form title
Value_score = rating / price

```
Data columns (total 14 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   asin              1448 non-null   object
 1   product_title     1448 non-null   object
 2   brand             1448 non-null   object
 3   price             1448 non-null   float64
 4   discount_percent  1448 non-null   float64
 5   rating            1448 non-null   float64
 6   review_count      1448 non-null   float64
 7   category          1448 non-null   object
 8   product_link      1448 non-null   object
 9   search_term       1448 non-null   object
 10  value_score       1448 non-null   float64
 11  device_type       1448 non-null   object
 12  is_wired          1448 non-null   int64
 13  has_mic           1448 non-null   int64
dtypes: float64(5), int64(2), object(7)
memory usage: 169.7+ KB
```

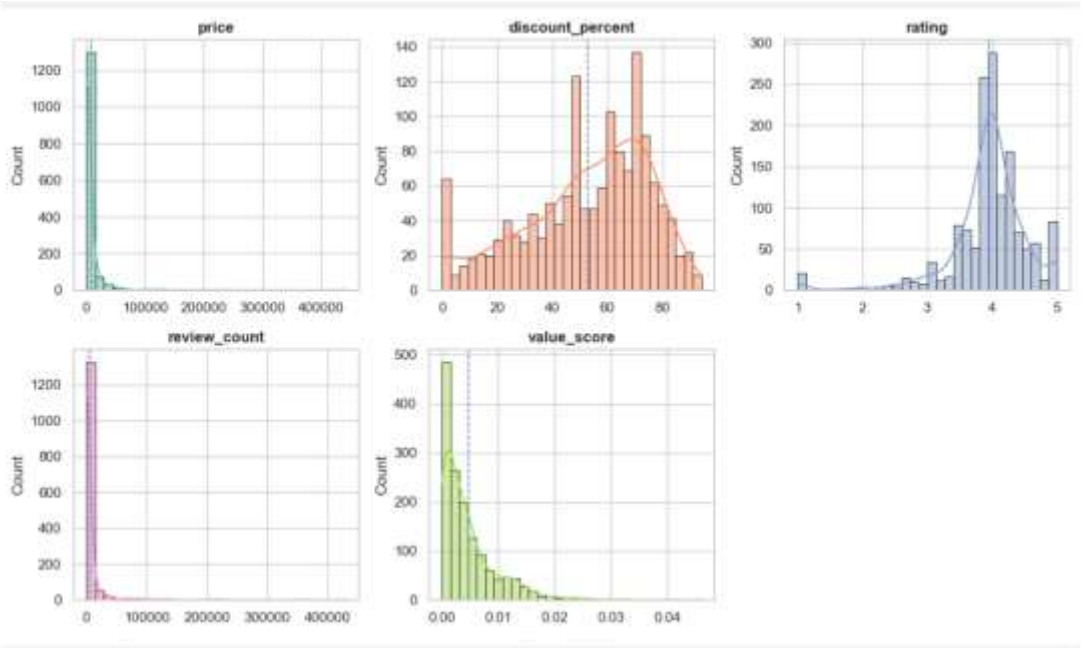
Exploratory Data Analysis (EDA)

Exploring:

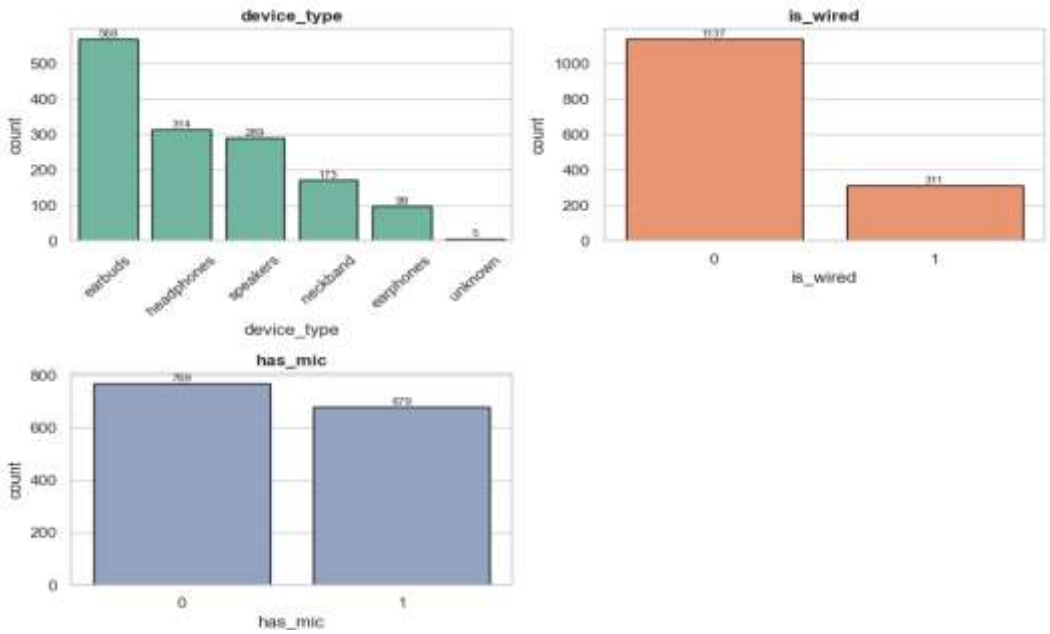
Isnull() , info() , describe() , nunique()

Key trends/visualization:

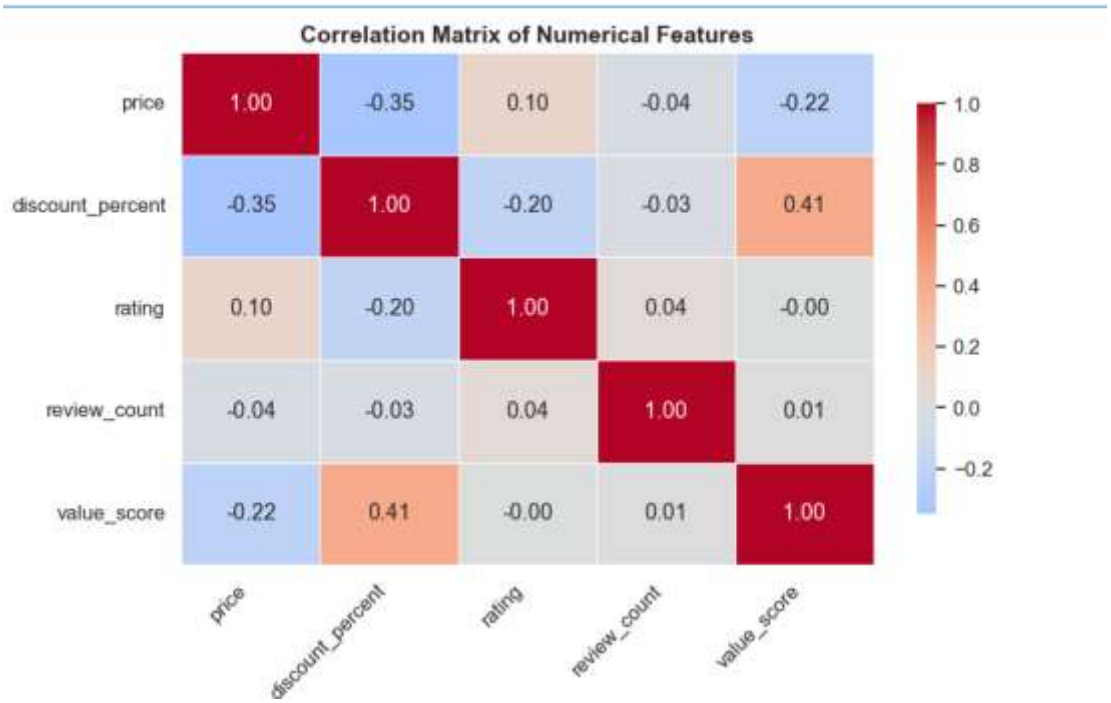
1) **Distribution graphs for all numeric column**



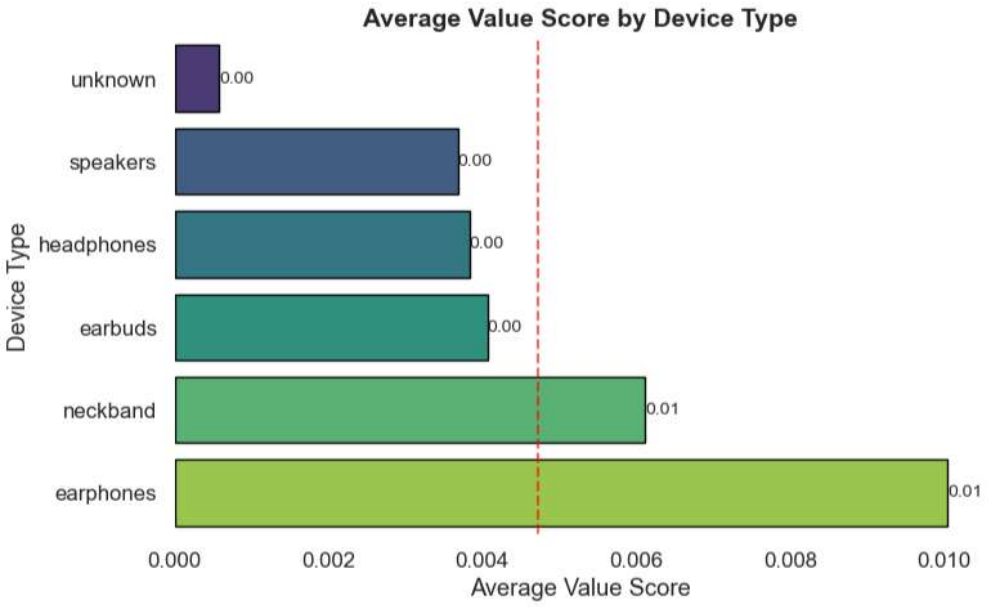
2) **Value count graphs for all categorical column**



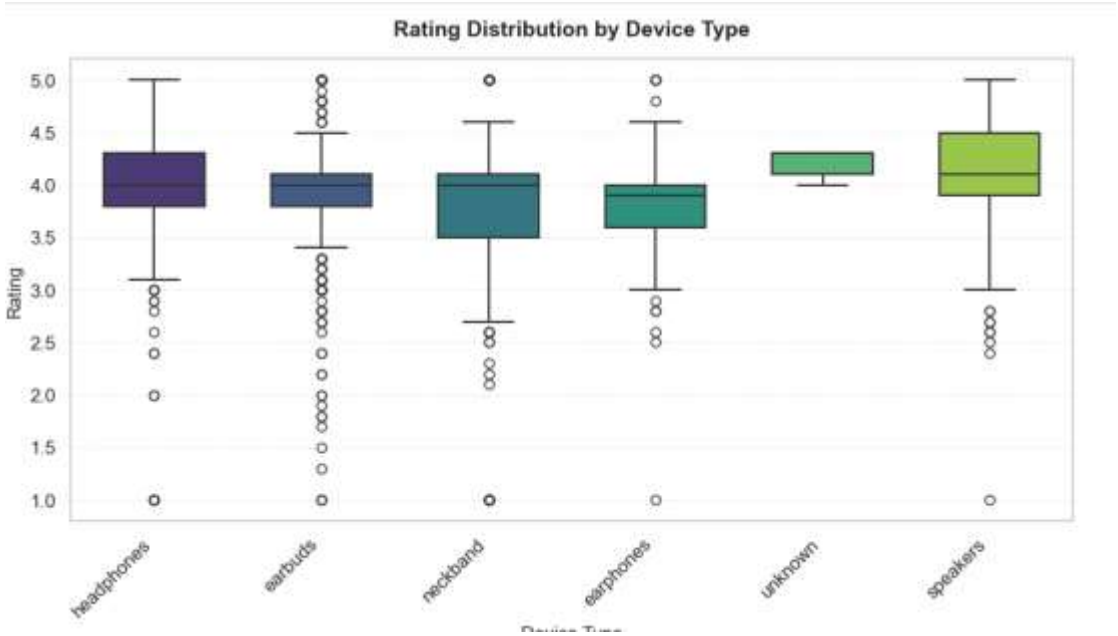
3) Correlation analysis



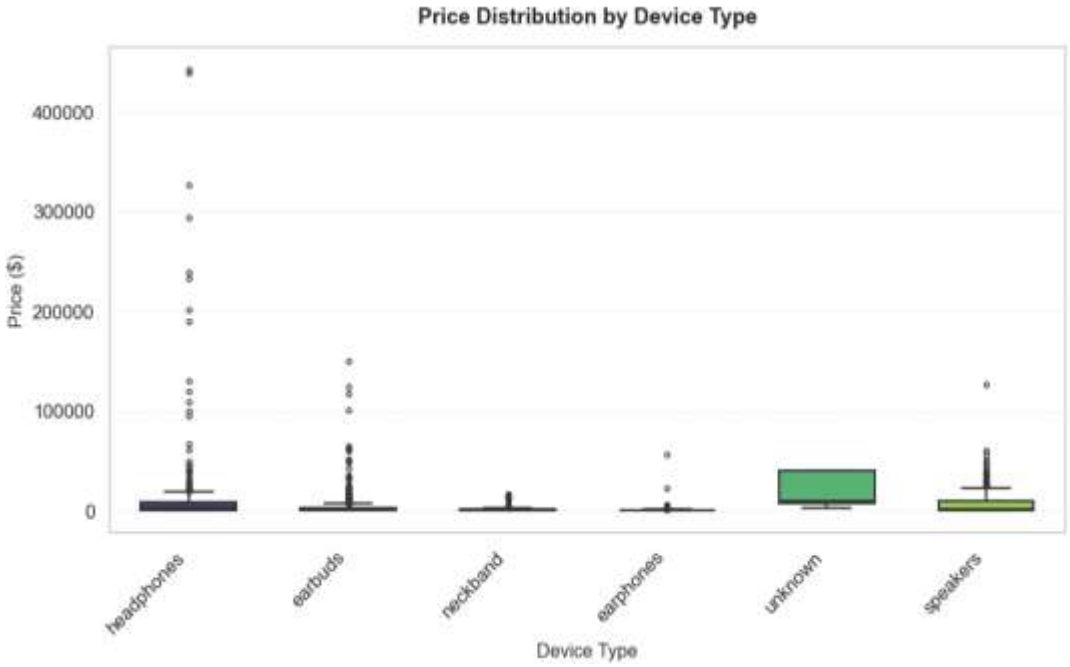
4) Average value score by each device type



5) **Rating distribution for each device type**



6) **Price distribution for each device type**

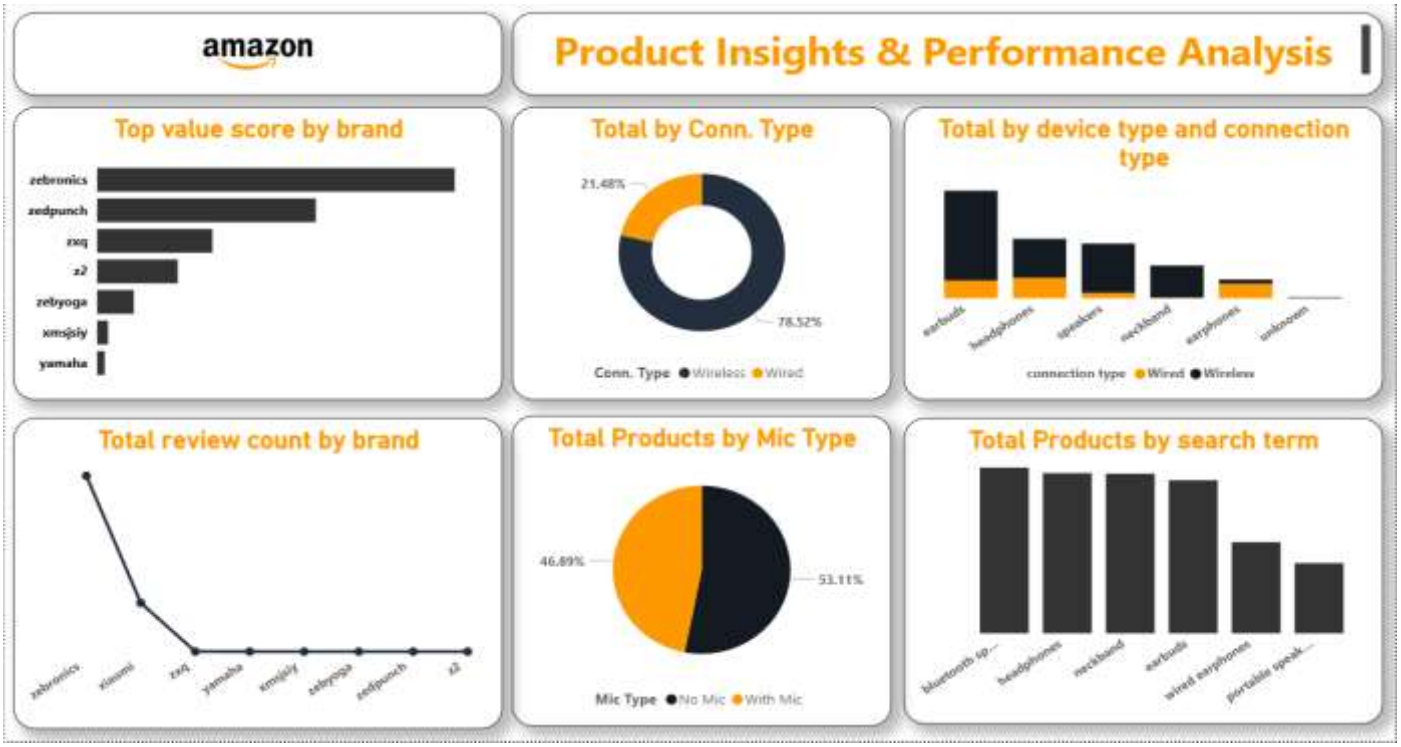


Dashboard Development

Tool: Power Bi

1) **Overview**

2) **Products features**



3) **Segmentation analysis**

4) **Recommendations and insights**

- **Insight: Zebronics dominates value score, followed by Zedpunch, showing strong price-to-performance positioning.**
  Recommendation: Focus sourcing or promotion on these brands—they offer the best competitive value mix.

- **Insight: Wireless products (~78%) heavily dominate the market over wired (~22%).**
  Recommendation: Prioritize wireless inventory and analysis—this is clearly where demand and growth lie.

- **Insight: "Bluetooth speakers," "headphones," and "neckband" are the mo st searched, showing demand concentration.**
  Recommendation: Optimize listings and analysis around top search terms to capture high-intent traffic.

- **Insight: Zebronics and Xiaomi have significantly higher review volumes, in dicating strong market trust and popularity.**
  Recommendation: Highlight high-review brands in recommendations—they drive higher customer confidence and conversions.

- **Insight: Products with mic (~53%) slightly exceed those without, showing p reference for multifunctional devices.**
  Recommendation: Prioritize "with mic" products—they align better with current user needs (calls, gaming, meetings).

- **Insight: Brands like boAt, JBL, and Portronics span across all segments, whi le Sony is more premium-focused.**
  Recommendation: Position multi-segment brands for volume strategy, and premium brands for high-margin targeting.

- **Insight: Most high ratings cluster in low-to-mid price range, with no strong correlation between high price and high rat ing.**
  Recommendation: Focus on mid-range pricing strategies—premium pricing doesn't guarantee better customer satisfaction.

- **Insight: Popular and Premium segments have more "with mic" products, w hile Low Rated has fewer mic-enabled devices.**
  Recommendation: Focus on mic-enabled products in higher segments—they align with user expectations and drive value perception.

- **Insight: Wireless dominates across all segments, especially in Popular and Premium categories.**
  Recommendation: Prioritize wireless products across segments—they are clearly the market standard.

- **Insight: Popular segment dominates (595 products), while Low Rated is sig nificantly smaller.**
  Recommendation: Target Popular segment for optimization—it has the highest impact on overall performance.

Machine Learning Models

1) Product Segmentation

Unsupervised machine learning
Objective: To cluster the products based on price, rating, and review count.

- **Log the extreme value to prevent ouliers**

```
df['price'] = np.log1p(df['price'])
df['review_count'] = np.log1p(df['review_count'])
```

- **Assigning the 'x' variables**

```
x = df[['price', 'rating', 'review_count' ]]
```

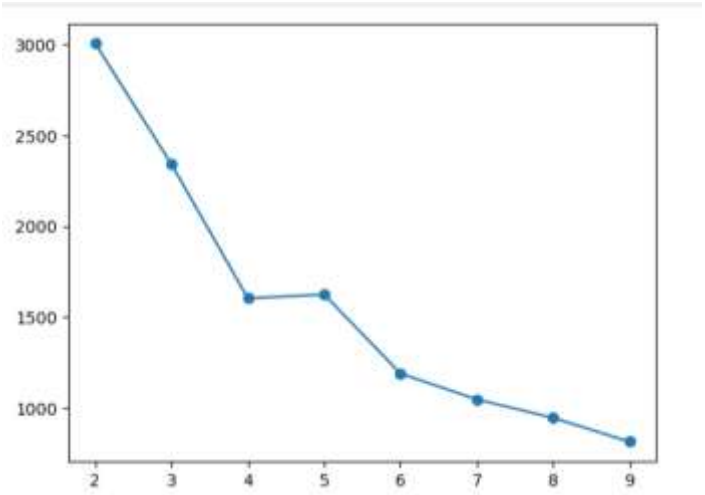
- **Standardization**

```
from sklearn.preprocessing import StandardScaler
sclaer = StandardScaler()
x_scaled = sclaer.fit_transform(x)
```

- **Calculating the wcss**

```
from sklearn.cluster import KMeans
wcss = []
for k in range(2,10):
   m = KMeans( n_clusters = k, random_state = 42 )
   m.fit(x_scaled)
   wcss.append(m.inertia_)
```

- **Plotting elbow curve**



- **Calculating silhouette score**

```
k = 2 Silhouette Score = 0.2990
k = 3 Silhouette Score = 0.3083
k = 4 Silhouette Score = 0.3587
k = 5 Silhouette Score = 0.2830
k = 6 Silhouette Score = 0.3241
k = 7 Silhouette Score = 0.3319
k = 8 Silhouette Score = 0.3136
k = 9 Silhouette Score = 0.3309
```

- **Optimal K = 4**

  Because we have elbow at 4th cluster and high silhouette score also.

  m1 = KMeans(n_clusters=4, random_state=42)
  clusters = m1.fit_predict(x_scaled)
- **Converting to data frame**
-

  df['cluster'] = clusters

  df.groupby('cluster')[['price','rating','review_count']].mean()

	price	rating	review_count
cluster			
0	6.537041	4.217403	1.565898
1	6.908696	3.859664	7.303750
2	9.452326	4.229581	6.374817
3	6.565967	2.366055	2.231570

- **Segmenting the cluster**
  segment_map = {
    0: "High Rated",
    1: "Popular",
    2: "Premium",
    3: "Low Rated"

```
}
df['segment'] = df['cluster'].map(segment_map)
```

```
df['segment'].value_counts()
```

```
segment
Popular       595
Premium       382
High Rated    362
Low Rated     109
Name: count, dtype: int64
```

Cluster 0: Top Rated
Good rating, Less reviews,Quality products people like.

Cluster 1 : Popular
Lots of reviews, Average rating, Frequently bought products.

Cluster 2 : Premium
Expensive, Good rating, High-end products.

luster 3 : Low Rated
Poor ratings, Not recommended.

2) Device Prediction

Supervised machine learning
Objective: To predict the device type from product title and is wired variables.

- **Encoding using Tfidf (NLP)**
  from sklearn.feature_extraction.text import TfidfVectorizer

  tfidf = TfidfVectorizer(stop_words = 'english', max_features = 5000, ngram_range=(1,2))
  x_product = tfidf.fit_transform(df['product_title'])

- **Assigning variables**
  x = hstack([x_product, x_iswired])
  y = df['device_type']

- **Spliting x, y**
  x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=42)

- **Modeling**
  model = LogisticRegression(max_iter=1000)
  model.fit(x_train, y_train)

- **Evaluation**

```
Accuracy: 0.8758620689655172
              precision    recall  f1-score   support

     earbuds       0.80      0.97      0.88       110
   earphones       0.79      0.65      0.71        17
  headphones       0.93      0.86      0.89        64
    neckband       0.96      0.69      0.81        36
    speakers       0.97      0.89      0.93        63

    accuracy                           0.88       290
   macro avg       0.89      0.81      0.84       290
weighted avg       0.89      0.88      0.87       290
```

Overall Metrics:

- **Precision (Weighted Avg): 0.89** → Predictions are generally accurate
- **Recall (Weighted Avg): 0.88** → Most actual products are correctly identified
- **F1-Score (Weighted Avg): 0.87** → Balanced overall performance

3) Content-Based Recommendation System

Unsupervised machine learning (NLP)
Objective: To give recommendation based on similar products.

- **Assigning the x varibles (Combining all together)**

```
df['combined_text'] = (
    df['product_title'] + " " +
    df['brand'] + " " +
    df['category'] + " " +
    df['device_type']
)
```

- **NLP (encoding the x variable)**

```
rom sklearn.feature_extraction.text import TfidfVectorizer
tfidf = TfidfVectorizer(stop_words='english', max_features=3000)
text_matrix = tfidf.fit_transform(df['combined_text'])
```

- **Applying Cosine Similarity**

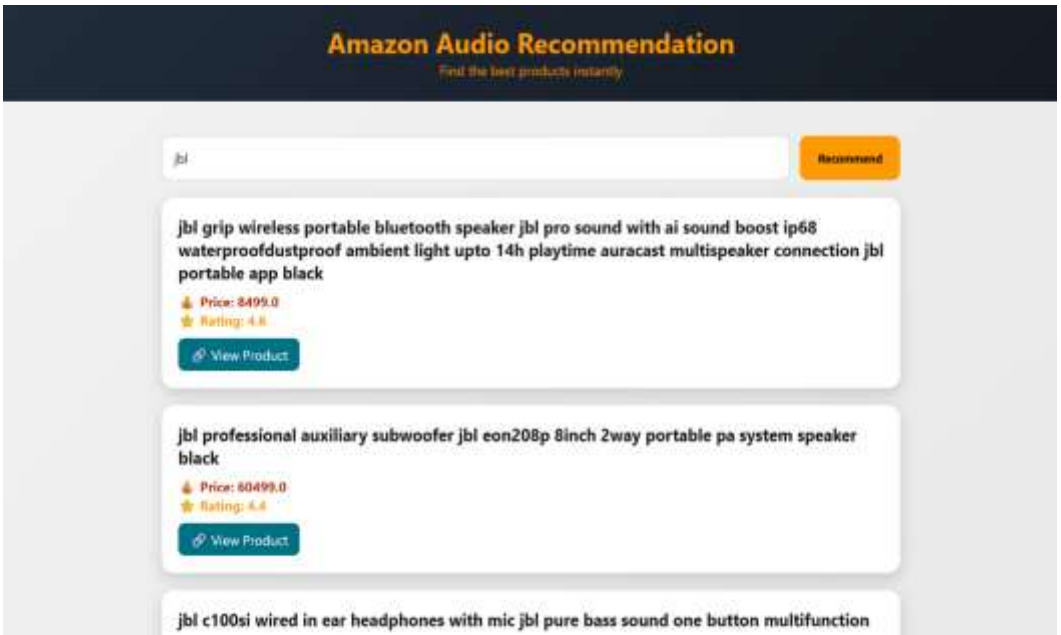
```
from sklearn.metrics.pairwise import cosine_similarity

def recommend_from_text(query, top_n=5, threshold=0.2):
    # Convert user query to vector
    query_vec = tfidf.transform([query])
    # Compute similarity
    sim_scores = cosine_similarity(query_vec, text_matrix).flatten()
    # Get sorted indices (highest similarity first)
    top_indices = sim_scores.argsort()[::-1]
    # Filter relevant results
```

```
    filtered = [i for i in top_indices if sim_scores[i] > threshold]
    # If nothing found
    if len(filtered) == 0:
        return None
    # Return top results
    return df[['product_title', 'price', 'rating', 'is_wired']].iloc[filtered[:top_n]]
```

- **Testing**
  recommend_from_text(" wireless ")

	product_title	price	rating	is_wired
200	wireless headset sh12 wireless bluetooth headp...	649.0	2.9	0
1258	compatible for wireless earbuds with charging ...	499.0	5.0	0
768	wireless bluetooth neckband with bluetooth 54 ...	427.0	4.1	0
754	wireless headphones neckband earphones in ear ...	499.0	5.0	0
1091	wireless earbudsbluetooth 54 headphones in ear...	6432.0	4.9	0

- **Implementing Website**



Video_link-
https://drive.google.com/file/d/1eqm1tfcfMgwD39zRmX2qX8OazRjHUr_C/view?usp=drive_link